

Nota Pengajaran Sains Komputer

Bab 1.3.4 - 1.4.3

Disusun semula sebagai handout guru: ringkas, visual, mudah diajar, serta disertakan contoh kod, jadual, aktiviti pantas, semakan sendiri, dan jawapan guru dalam warna merah.

Topik utama	1.3.4	Pengisytiharan dan penetapan nilai pemboleh ubah/pemalar.
	1.3.5	Penyataan umpukan dan aritmetik.
	1.3.6	Input papan kekunci dan output.
	1.4.1	Struktur kawalan pilihan.
	1.4.2	Operator hubungan dan operator logikal.
	1.4.3	Struktur kawalan ulangan.

Isi Kandungan

Panduan Penggunaan Nota	2
1 1.3.4 Pengisytiharan, Permulaan dan Penetapan Nilai	3
2 1.3.5 Penyataan Umpukan dan Penyataan Aritmetik	5
3 1.3.6 Input Papan Kekunci dan Paparan Output	7
4 1.4.1 Struktur Kawalan Pilihan	9
5 1.4.2 Operator Hubungan dan Operator Logikal	11
6 1.4.3 Struktur Kawalan Ulangan	13
Ringkasan Cepat Bab 1.3.4 - 1.4.3	15
Latihan Pengukuhan	16
Skema Jawapan Ringkas Untuk Guru	17

Panduan Penggunaan Nota

Objektif Pembelajaran

Selepas mempelajari nota ini, murid sepatutnya boleh:

1. mengisytiharkan pemboleh ubah dan pemalar dengan jenis data yang sesuai;
2. menulis pernyataan umpukan dan pernyataan aritmetik dalam Java;
3. menggunakan Scanner untuk menerima input daripada papan kekunci;
4. membina struktur kawalan pilihan menggunakan if, if-else, if-else-if dan switch-case;
5. menggunakan operator hubungan dan logikal dalam ungkapan syarat;
6. membezakan gelung for, while dan do-while.

Nota Guru: Cadangan aliran kelas 60 minit

- | | |
|------------------|--|
| 0-10 min | Ulang kaji konsep data, pemboleh ubah dan jenis data. |
| 10-25 min | Demonstrasi pengisytiharan, umpukan dan aritmetik menggunakan contoh pendek. |
| 25-38 min | Demonstrasi input/output dengan Scanner. |
| 38-50 min | Bina keputusan menggunakan if-else dan operator hubungan/logikal. |
| 50-60 min | Aktiviti pantas: murid lengkapkan kod mini dan jawab semakan sendiri. |

Aktiviti Murid: Cara belajar aktif

Semasa guru menerangkan, murid perlu menulis sendiri tiga perkara pada ruang kosong:

- satu contoh pemboleh ubah: `int umur = 16;`
- satu contoh syarat boolean: `markah >= 40`
- satu contoh gelung: `for (int i = 1; i <= 5; i++)`

1 1.3.4 Pengisytiharan, Permulaan dan Penetapan Nilai

Idea Utama

Dalam Java, setiap pemboleh ubah perlu **diisytiharkan** dengan jenis data tertentu sebelum digunakan. Ini memberitahu komputer jenis maklumat yang akan disimpan, contohnya nombor bulat, nombor perpuluhan, teks, aksara atau nilai benar/palsu.

Pemboleh ubah vs pemalar

Konsep	Maksud	Contoh dunia sebenar
Pemboleh ubah	Ruang simpanan sementara yang nilainya boleh berubah semasa program berjalan.	Markah murid, umur pengguna, jumlah harga.
Pemalar	Nilai tetap yang tidak berubah sepanjang program berjalan.	Nilai Pi, bilangan hari dalam seminggu, kadar cukai tetap.

Sintaks: Pengisytiharan dan permulaan nilai

```
1 jenisData namaPembolehUbah;
2 jenisData namaPembolehUbah = nilai;
3 final jenisData NAMA_PEMALAR = nilai;
```

Contoh asas dalam Java

```
1 int umur = 16;
2 String nama = "Hardeep";
3 double tinggi = 1.68;
4 boolean lulus = true;
5 char gred = 'A';
6 final double PI = 3.142;
```

Jadual jenis data yang lazim

Jenis Data	Tujuan	Contoh sintaks Java
int	Menyimpan nombor bulat.	int x = 10;
String	Menyimpan teks atau perkataan.	String nama = "Hardeep";
double	Menyimpan nombor perpuluhan.	double s = 0.123;
boolean	Menyimpan nilai true atau false.	boolean v = true;
char	Menyimpan satu aksara sahaja.	char w = 'y';

Awat: Kesilapan lazim

- String menggunakan tanda petik dua: "Ali".
- char menggunakan tanda petik satu: 'A'.
- Nama pemalar biasanya ditulis dengan huruf besar, contohnya PI atau HARI_SEMINGGU.
- Java peka huruf besar dan kecil: markah dan Markah dianggap nama yang berbeza.

Aktiviti Murid: Lengkapi kod

Isikan jenis data yang sesuai.

```
1 int umur = 15;  
2 String nama = "Mei Ling";  
3 double suhu = 36.5;  
4 boolean hadir = true;  
5 char gred = 'B';
```

2 1.3.5 Penyataan Umpukan dan Penyataan Aritmetik

Idea Utama

Penyataan umpukan menggunakan simbol `=` untuk memasukkan nilai ke dalam pemboleh ubah. Penyataan aritmetik pula menggunakan operator matematik seperti tambah, tolak, darab dan bahagi untuk menghasilkan nilai baharu.

Penyataan umpukan

```
1 int markah = 85;           // memasukkan nilai 85 ke dalam markah
2 double harga = 12.50;     // memasukkan nilai 12.50 ke dalam harga
3 markah = 90;              // nilai markah ditukar daripada 85 kepada 90
```

Nota Guru: Bezakan `=` dan `==`

Simbol `=` bermaksud *assign* atau memasukkan nilai. Simbol `==` bermaksud membandingkan sama ada dua nilai adalah sama. Ini sangat penting sebelum murid belajar struktur kawalan pilihan.

Operator aritmetik

Simbol Java	Simbol Matematik	Penerangan
<code>+</code>	<code>+</code>	Operasi tambah.
<code>-</code>	<code>-</code>	Operasi tolak.
<code>*</code>	<code>×</code>	Operasi darab.
<code>/</code>	<code>÷</code>	Operasi bahagi.
<code>%</code>	baki bahagi	Menghasilkan baki selepas pembahagian. Contoh: <code>10 % 3</code> menghasilkan 1.

Awat: Keutamaan operator

Java tidak menjalankan semua operasi hanya dari kiri ke kanan. Java mengikut **keutamaan operator**:

- `()` dilaksanakan dahulu;
- `*`, `/`, `%` dilaksanakan sebelum `+` dan `-`;
- operator yang sama tahap biasanya dinilai dari kiri ke kanan.

Contoh pengiraan

```
1 int a = 10;
2 int b = 3;
3
```

```
4 int tambah = a + b;           // 13
5 int tolak = a - b;           // 7
6 int darab = a * b;           // 30
7 int bahagi = a / b;          // 3, kerana int / int menghasilkan int
8 int baki = a % b;            // 1
9
10 double tepat = 10.0 / 3;    // 3.333...
```

Aktiviti Murid: Ramalkan output

Apakah nilai bagi jawapan?

```
1 int jawapan = 4 + 2 * 3;
```

Jawapan guru: **10**

3 1.3.6 Input Papan Kekunci dan Paparan Output

Idea Utama

Program yang interaktif membolehkan pengguna memasukkan data semasa program sedang berjalan. Dalam Java, kelas Scanner digunakan untuk membaca input daripada papan kekunci.

Tiga langkah menggunakan Scanner

1. Import kelas Scanner di bahagian atas program.
2. Cipta objek Scanner untuk membaca input.
3. Gunakan kaedah input seperti `nextInt()`, `nextDouble()` atau `nextLine()`.

Sintaks: Input dan output

```
1 import java.util.Scanner;
2
3 Scanner input = new Scanner(System.in);
4 int umur = input.nextInt();
5 System.out.println("Umur anda ialah " + umur);
```

Kaedah input yang biasa digunakan

Kaedah	Digunakan untuk	Contoh
<code>nextInt()</code>	Nombor bulat.	<code>int umur = input.nextInt();</code>
<code>nextDouble()</code>	Nombor perpuluhan.	<code>double berat = input.nextDouble();</code>
<code>next()</code>	Satu perkataan tanpa ruang.	<code>String nama = input.next();</code>
<code>nextLine()</code>	Satu baris teks lengkap.	<code>String alamat = input.nextLine();</code>

Contoh program lengkap

```
1 import java.util.Scanner;
2
3 public class ProfilMurid {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6
7         System.out.print("Masukkan nama: ");
8         String nama = input.nextLine();
9
10        System.out.print("Masukkan umur: ");
11        int umur = input.nextInt();
```



```
12
13     System.out.println("Hai " + nama + "!");
14     System.out.println("Umur anda ialah " + umur + " tahun.");
15 }
16 }
```

Nota Guru: Masalah biasa nextLine()

Jika `nextLine()` digunakan selepas `nextInt()` atau `nextDouble()`, Java mungkin membaca baki baris kosong. Penyelesaian mudah: tambah satu `input.nextLine();` selepas membaca nombor untuk membuang baki baris tersebut.

Aktiviti Murid: Bina output sendiri

Tulis satu pernyataan `System.out.println()` yang memaparkan nama dan markah.

```
1 String nama = "Aina";
2 int markah = 92;
3
4 System.out.println("Nama: " + nama + ", Markah: " + markah);
```

4 1.4.1 Struktur Kawalan Pilihan

Idea Utama

Struktur kawalan pilihan membolehkan program membuat keputusan secara automatik berdasarkan syarat boolean. Jika syarat bernilai `true`, satu blok arahan akan dilaksanakan; jika `false`, blok lain boleh dilaksanakan.

Empat bentuk struktur kawalan pilihan

Struktur	Kegunaan
<code>if</code>	Melaksanakan arahan hanya apabila syarat benar.
<code>if-else</code>	Memberi dua laluan keputusan: benar atau palsu.
<code>if-else-if</code>	Menguji beberapa syarat secara berturutan.
<code>switch-case</code>	Memadankan satu nilai dengan beberapa pilihan kes tertentu.

Contoh `if` dan `if-else`

```
1 int markah = 75;
2
3 if (markah >= 40) {
4     System.out.println("Lulus");
5 } else {
6     System.out.println("Gagal");
7 }
```

Contoh `if-else-if`

```
1 int markah = 82;
2
3 if (markah >= 90) {
4     System.out.println("Gred A+");
5 } else if (markah >= 80) {
6     System.out.println("Gred A");
7 } else if (markah >= 60) {
8     System.out.println("Gred B");
9 } else {
10    System.out.println("Perlu cuba lagi");
11 }
```

Contoh switch-case

```
1 int hari = 3;
2
3 switch (hari) {
4     case 1:
5         System.out.println("Isnin");
6         break;
7     case 2:
8         System.out.println("Selasa");
9         break;
10    case 3:
11        System.out.println("Rabu");
12        break;
13    default:
14        System.out.println("Nombor hari tidak sah");
15 }
```

Awas: Jangan lupa break

Dalam switch-case, break digunakan untuk keluar daripada blok switch selepas kes yang sepadan dijalankan. Tanpa break, program boleh terus menjalankan kes seterusnya.

5 1.4.2 Operator Hubungan dan Operator Logikal

Idea Utama

Operator hubungan membandingkan dua nilai dan menghasilkan keputusan boolean. Operator logikal pula menggabungkan dua atau lebih syarat boolean untuk membina keputusan yang lebih kompleks.

Operator hubungan

Simbol	Maksud	Contoh
<code>==</code>	Sama dengan.	<code>markah == 100</code>
<code>!=</code>	Tidak sama dengan.	<code>jawapan != 0</code>
<code>></code>	Lebih besar daripada.	<code>umur > 18</code>
<code>>=</code>	Lebih besar atau sama dengan.	<code>markah >= 40</code>
<code><</code>	Kurang daripada.	<code>suhu < 37.5</code>
<code><=</code>	Kurang atau sama dengan.	<code>markah <= 100</code>

Operator logikal

Simbol	Nama	Maksud
<code>&&</code>	AND	Semua syarat mesti benar untuk menghasilkan true.
<code> </code>	OR	Sekurang-kurangnya satu syarat mesti benar untuk menghasilkan true.
<code>!</code>	NOT	Menukarkan true kepada false, atau false kepada true.

Contoh gabungan syarat

```

1 int markah = 85;
2
3 if (markah >= 0 && markah <= 100) {
4     System.out.println("Markah sah");
5 } else {
6     System.out.println("Markah tidak sah");
7 }

```

Nota Guru: Truth table mini

A	B	A && B	A B	!A
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

Aktiviti Murid: Kenal pasti benar atau palsu

Diberi `int` `markah = 75`; dan `boolean` `hadir = true`;

1. `markah >= 40`

Jawapan: **true**

2. `markah > 80`

Jawapan: **false**

3. `markah >= 40 && hadir == true`

Jawapan: **true**

4. `markah > 80 || hadir == true`

Jawapan: **true**

6 1.4.3 Struktur Kawalan Ulangan

Idea Utama

Struktur kawalan ulangan membolehkan satu blok arahan diulang beberapa kali. Ulangan boleh berdasarkan bilangan tertentu atau berdasarkan syarat boolean yang masih benar.

Perbandingan for, while dan do-while

Jenis gelung	Bila digunakan?	Sintaks ringkas
for	Apabila bilangan ulangan diketahui atau mudah dikira.	for (pemula; syarat; penambah) { ... }
while	Apabila ulangan bergantung kepada syarat, dan mungkin tidak berlaku langsung.	while (syarat) { ... }
do-while	Apabila blok arahan mesti dijalankan sekurang-kurangnya sekali.	do { ... } while (syarat);

Contoh for

```
1 for (int i = 1; i <= 5; i++) {  
2     System.out.println("Ulangan ke-" + i);  
3 }
```

Contoh while

```
1 int i = 1;  
2  
3 while (i <= 5) {  
4     System.out.println("Ulangan ke-" + i);  
5     i++;  
6 }
```

Contoh do-while

```
1 int i = 1;  
2  
3 do {  
4     System.out.println("Ulangan ke-" + i);  
5     i++;  
6 } while (i <= 5);
```

Awas: Gelung infiniti

Gelung infiniti berlaku apabila syarat gelung sentiasa benar. Pastikan nilai pembilang atau syarat boolean dikemas kini dalam gelung.

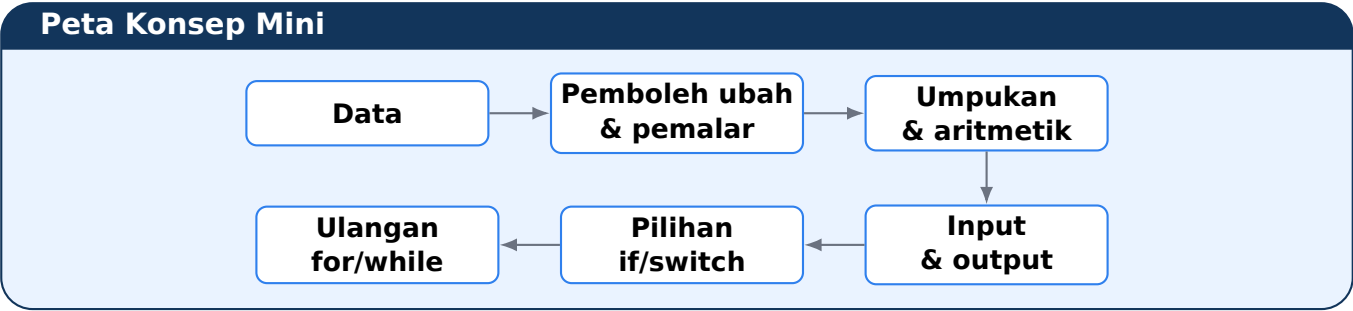
```
1 int i = 1;
2 while (i <= 5) {
3     System.out.println(i);
4     // i++ terlupa: gelung tidak akan berhenti
5 }
```

Aktiviti Murid: Pilih gelung yang sesuai

Tulis for, while atau do-while.

1. Cetak nombor 1 hingga 10. **for**
2. Ulang selagi kata laluan pengguna salah. **while**
3. Papar menu sekurang-kurangnya sekali sebelum pengguna memilih keluar. **do-while**

Ringkasan Cepat Bab 1.3.4 - 1.4.3



Topik	Ingatan utama
Pemboleh ubah	Mesti ada jenis data sebelum digunakan. Contoh: <code>int umur = 16;</code>
Pemalar	Gunakan <code>final</code> . Contoh: <code>final double PI = 3.142;</code>
Umpukan	<code>=</code> memasukkan nilai; <code>==</code> membandingkan nilai.
Aritmetik	<code>*</code> , <code>/</code> , <code>%</code> lebih tinggi keutamaannya daripada <code>+</code> dan <code>-</code> .
Input	Gunakan <code>import java.util.Scanner;</code> dan objek <code>Scanner</code> .
Output	Gunakan <code>System.out.print()</code> atau <code>System.out.println()</code> .
Pilihan	Gunakan <code>if</code> , <code>if-else</code> , <code>if-else-if</code> , <code>switch-case</code> .
Logikal	<code>&&</code> semua benar, <code> </code> salah satu benar, <code>!</code> lawan boolean.
Ulangan	<code>for</code> untuk bilangan diketahui, <code>while</code> untuk syarat, <code>do-while</code> sekurang-kurangnya sekali.

Latihan Pengukuhan

Aktiviti Murid: Bahagian A: Lengkapi tempat kosong

1. Jenis data untuk nombor bulat ialah **int**.
2. Simbol untuk operasi darab dalam Java ialah *****.
3. Untuk membaca input daripada papan kekunci, kita perlu mengimport **java.util.Scanner**.
4. Operator logikal AND ditulis sebagai **&&**.
5. Gelung yang akan berjalan sekurang-kurangnya sekali ialah **do-while**.

Aktiviti Murid: Bahagian B: Betulkan kod

Kenal pasti dan betulkan kesilapan berikut.

```
1 int markah = "90";  
2 char gred = "A";  
3 if (markah = 90) {  
4     System.out.println("Cemerlang")  
5 }
```

Ruang jawapan murid / jawapan guru:

```
1 int markah = 90;  
2 char gred = 'A';  
3  
4 if (markah == 90) {  
5     System.out.println("Cemerlang");  
6 }
```

Aktiviti Murid: Bahagian C: Cabaran mini program

Tulis program Java yang:

1. meminta pengguna memasukkan nama dan markah;
2. menyemak sama ada markah berada antara 0 hingga 100;
3. memaparkan "Lulus" jika markah 40 atau lebih;
4. memaparkan "Gagal" jika markah kurang daripada 40;
5. menggunakan Scanner, if-else dan operator logikal.

Skema Jawapan Ringkas Untuk Guru

Nota Guru: Jawapan aktiviti utama

1.3.4 Lengkapkan kod

```
1 int umur = 15;  
2 String nama = "Mei Ling";  
3 double suhu = 36.5;  
4 boolean hadir = true;  
5 char gred = 'B';
```

1.3.5 Ramalkan output

4 + 2 * 3 = 10 kerana darab dilaksanakan sebelum tambah.

1.4.2 Benar atau palsu

1. **true**; 2. **false**; 3. **true**; 4. **true**.

1.4.3 Pilih gelung

1. **for**; 2. **while**; 3. **do-while**.

Nota Guru: Cadangan jawapan Bahagian B

```
1 int markah = 90;  
2 char gred = 'A';  
3  
4 if (markah == 90) {  
5     System.out.println("Cemerlang");  
6 }
```

Nota Guru: Cadangan jawapan Cabaran mini program

```
1 import java.util.Scanner;
2
3 public class SemakMarkah {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6
7         System.out.print("Masukkan nama: ");
8         String nama = input.nextLine();
9
10        System.out.print("Masukkan markah: ");
11        int markah = input.nextInt();
12
13        if (markah >= 0 && markah <= 100) {
14            if (markah >= 40) {
15                System.out.println(nama + " lulus.");
16            } else {
17                System.out.println(nama + " gagal.");
18            }
19        } else {
20            System.out.println("Markah tidak sah.");
21        }
22    }
23 }
```